

AI PDF 文档问答助手 - 中文练习资料

版本：v1.0 | 用途：RAG 入门项目测试 PDF

一、项目背景

本资料用于练习 RAG，也就是检索增强生成。RAG 的基本思想是：用户提出问题后，系统先从文档中检索相关内容，再把相关内容交给大语言模型生成答案。

这个练习项目的目标不是做一个复杂系统，而是帮助初学者理解 PDF 读取、文本切块、向量化、相似度检索和基于上下文回答这几个核心步骤。

如果大模型没有拿到文档内容，它通常无法准确回答 PDF 中的具体细节。因此，文档检索是这个项目最重要的部分之一。

二、学习目标

完成本项目后，学习者应该能够读取 PDF 文本，理解什么是文本切块，知道 Embedding 的作用，并能用简单的相似度检索找到相关段落。

学习者还应该能够设计一个基础提示词，让大模型只根据提供的上下文回答问题。如果上下文中没有答案，模型应该回答“文档中没有找到相关信息”。

本项目不要求学习者一开始就掌握复杂框架，例如 LangChain、LlamaIndex 或完整的向量数据库服务。建议先用最小代码理解原理。

三、推荐技术栈

Python 是本项目的主要编程语言，因为它有丰富的 PDF 处理、文本处理和 AI 开发库。

PDF 读取可以使用 `pypdf`。文本切块可以先自己写简单函数。Embedding 可以使用本地模型或云端 API。向量检索可以先使用 `NumPy` 计算余弦相似度。

当理解基本流程后，可以再升级到 Chroma、FAISS 或其他向量数据库。前端界面可以后续使用 Streamlit 或 Gradio 实现。

四、项目流程

第一步：用户上传 PDF 文件。第二步：程序读取 PDF 中的文字内容。第三步：将长文本切成多个较短的文本块。

第四步：把每个文本块转换成向量。第五步：用户输入问题后，也把问题转换成向量。第六步：计算问题向量和文本块向量之间的相似度。

第七步：选出最相关的几个文本块，作为上下文交给大语言模型。第八步：大语言模型基于上下文生成最终答案。

阶段	说明
读取	从 PDF 中提取文字内容
切块	把长文本拆成较短的片段
向量化	把文本转换成方便比较的数字向量
检索	根据用户问题找到最相关的文本块
回答	让大模型基于上下文生成答案

五、切块规则建议

对于入门项目，可以把每个文本块控制在 300 到 500 个中文字之间。文本块太短可能缺少上下文，文本块太长可能包含太多无关内容。

可以设置少量重叠内容，例如每个块之间保留 50 个字。这样做可以减少重要句子被切断后造成的信息丢失。

更高级的切块方式可以按照标题、段落、页码或语义边界来切，但入门阶段不需要一开始就做得太复杂。

六、回答规则

AI 回答时必须优先依据检索到的上下文，不应该凭空编造文档中没有的信息。

如果问题的答案不在文档中，系统应该明确说明：文档中没有找到相关信息。

如果答案来自多个段落，可以先综合总结，再列出依据。对于学习项目，可以在回答后显示引用的文本块编号，方便检查检索是否正确。

七、示例问题

问题一：这个项目的主要目标是什么？

问题二：为什么不能每次都把整份 PDF 直接交给大模型？

问题三：文本切块建议控制在多少中文字？

问题四：如果文档中没有答案，AI 应该怎么回答？

问题五：入门阶段推荐先用什么方式做向量检索？

八、常见错误

错误一：只读取了 PDF 第一页，导致后面内容完全没有进入知识库。

错误二：切块太大，检索结果包含很多无关内容。

错误三：没有检查 `extract_text` 的结果，导致空文本也被继续处理。

错误四：提示词没有限制模型必须根据上下文回答，导致模型出现编造内容。

错误五：只看最终答案是否流畅，却不检查检索到的文本块是否真的相关。

九、项目验收标准

一个合格的入门版 PDF 问答助手，至少应该能够完成以下任务：读取 PDF 文本、完成文本切块、保存文本块、根据问题检索相关文本、生成基于文档的回答。

当用户询问“文本切块建议控制在多少中文字”时，系统应该能找到第五部分的内容，并回答 300 到 500 个中文字。

当用户询问“作者最喜欢的电影是什么”时，由于文档没有相关信息，系统应该回答文档中没有找到相关信息，而不是随便猜测。

附注：这份 PDF 是专门为 RAG 入门学习生成的中文测试文档。你可以用它测试 PDF 文本提取、切块、检索和问答。